

EC2 Fleet Standardization Automation

Standardizing EC2 Infrastructure Changes Through Safe and Controlled Automation

Executive Summary

Infrastructure changes are among the highest-risk operational activities within any cloud environment. While resizing an EC2 instance is technically straightforward, executing the same change consistently across an entire fleet requires far more than simply calling the AWS API.

I built this automation to standardize the upgrade of fifteen Amazon EC2 instances from `t3.micro` to `t3.small` using Python and AWS Boto3. My objective was to design an automation workflow that prioritizes operational safety, repeatability, and visibility rather than simply completing the resize operation.

The solution incorporates automated discovery, configuration validation, safety checks, canary deployment, controlled batch execution, post-change verification, and structured reporting. Every stage is intentionally isolated to reduce operational risk and simplify troubleshooting should an unexpected failure occur.

Unlike one-off infrastructure scripts, this project was designed around the principles I consider essential for managing production infrastructure changes: validating assumptions before execution, minimizing blast radius, maintaining detailed execution records, and ensuring every change is deterministic and repeatable.

Business Problem

Managing infrastructure changes manually becomes increasingly inefficient as cloud environments expand. Even a relatively small fleet of EC2 instances requires engineers to repeatedly perform the same sequence of operational tasks while ensuring every server reaches the desired state.

The environment for this project consisted of fifteen EC2 instances that required standardization from **`t3.micro`** to **`t3.small`**. Although resizing a single instance requires only a few API calls, repeating the process manually across the fleet introduces unnecessary operational risks, including inconsistent execution, human error, longer maintenance windows, and limited visibility into completed changes.

Rather than creating another EC2 resize script, I focused on designing an automation workflow that approaches infrastructure changes as controlled engineering operations with validation, phased deployment, reporting, and operational safeguards built into every stage.

Solution Overview

Instead of treating the resize operation as a single execution step, I separated the workflow into independent phases that each perform a specific responsibility.

The automation follows the sequence below:

- Authentication
- Fleet Discovery

- Configuration Validation
- Safety Validation
- Canary Deployment
- Batch Processing
- Post-Execution Verification
- Report Generation

This separation improves maintainability, simplifies troubleshooting, and allows failures to be isolated before infrastructure modifications begin.

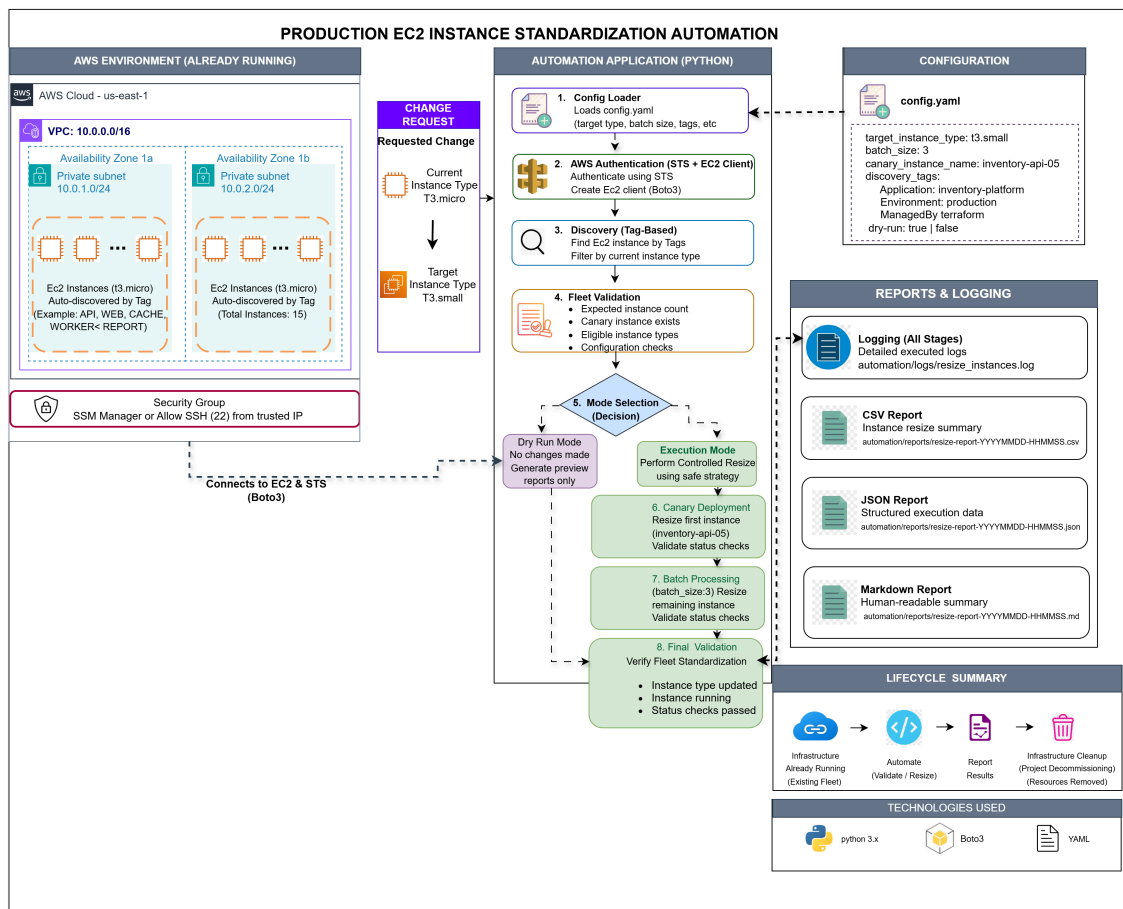


Figure 1. High-level architecture illustrating the end-to-end automation workflow from authentication through reporting.

Project Architecture

The solution follows a modular architecture where each component performs a single responsibility. Rather than placing all execution logic inside one script, I separated discovery, validation, execution, reporting, and infrastructure provisioning into independent modules.

This approach improves readability, simplifies testing, and makes future enhancements significantly easier to implement.

The project combines Infrastructure as Code with operational automation.

Terraform provisions the required AWS infrastructure, while Python orchestrates the EC2 standardization workflow using AWS Boto3.

This separation ensures infrastructure provisioning remains independent from operational change execution.

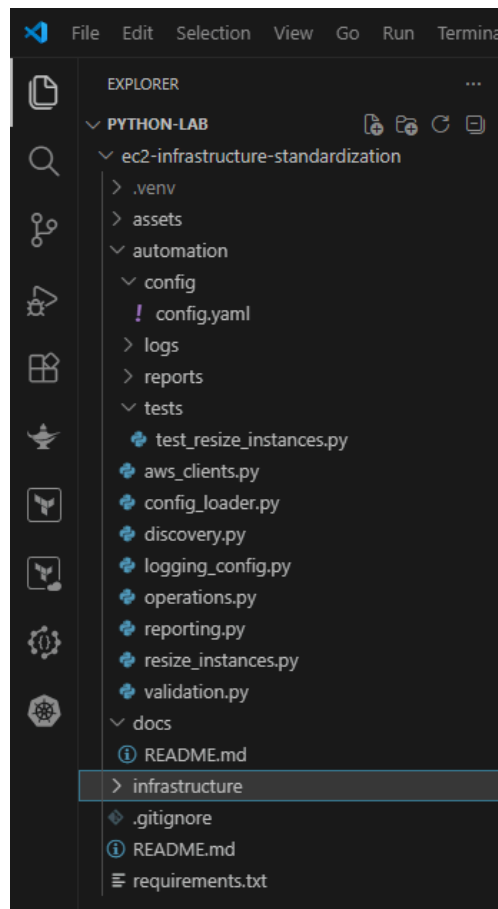
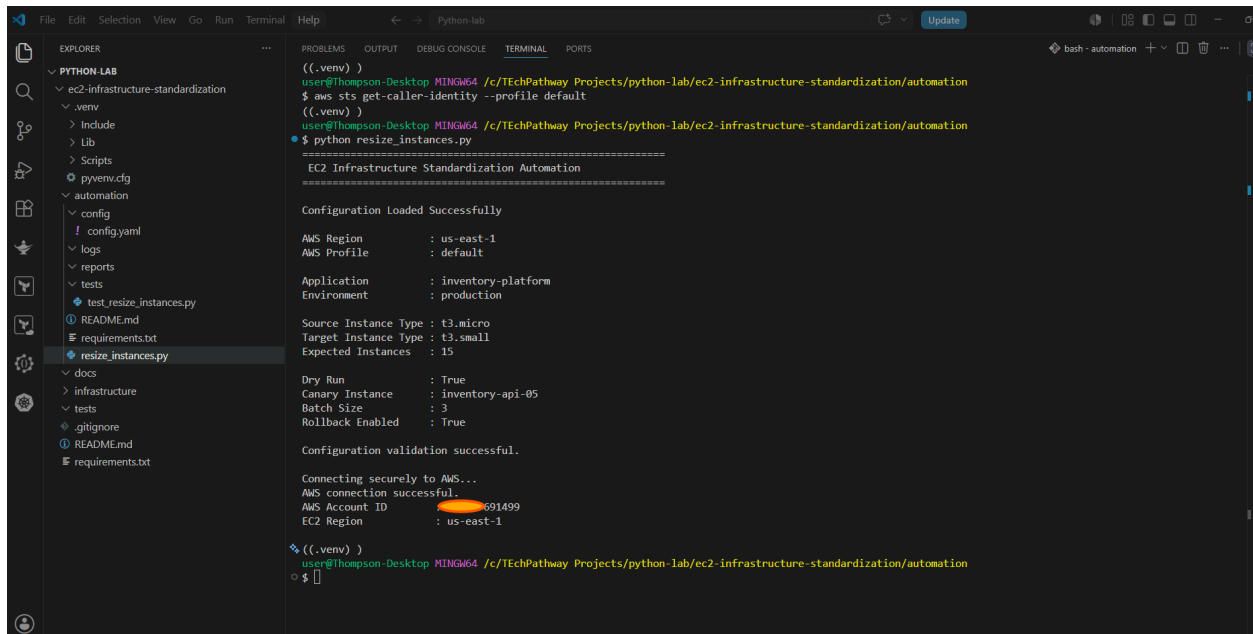


Figure 2. Repository organization showing the separation between infrastructure provisioning, automation modules, configuration, and reporting.

Authentication

Before making any infrastructure changes, the automation validates AWS authentication and confirms that the executing identity has sufficient permissions to perform the required operations.

Failing early during authentication prevents partially executed workflows caused by invalid credentials or insufficient IAM permissions.



```
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ aws sts get-caller-identity --profile default
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py
=====
EC2 Infrastructure Standardization Automation
=====

Configuration Loaded Successfully

AWS Region      : us-east-1
AWS Profile     : default

Application     : inventory-platform
Environment     : production

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances  : 15

Dry Run        : True
Canary Instance : inventory-api-05
Batch Size     : 3
Rollback Enabled : True

Configuration validation successful.

Connecting securely to AWS...
AWS connection successful.
AWS Account ID : 691499
EC2 Region     : us-east-1

((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$
```

Figure 3. Successful authentication and AWS identity verification before execution begins.

Configuration Validation

Infrastructure automation should never assume that configuration files are correct. Before discovering any EC2 instances, the automation validates the configuration file to ensure all required parameters are present and internally consistent. Detecting configuration errors at this stage prevents unnecessary API calls and eliminates avoidable execution failures later in the workflow.

```
def display_configuration(config):
    print(f"\nDry Run : {config['execution']['dry_run']}")

((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

=====
EC2 Infrastructure Standardization Automation
=====

Configuration Loaded Successfully

AWS Region      : us-east-1
AWS Profile     : default

Application     : inventory-platform
Environment     : production

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances  : 15

Dry Run        : True
Canary Instance : inventory-api-05
Batch Size     : 3
Rollback Enabled : True

Configuration validation successful.

((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$
```

Figure 4. Configuration successfully loaded and validated before infrastructure discovery.

Fleet Discovery

Rather than relying on manually maintained instance lists, the automation dynamically discovers EC2 instances based on predefined selection criteria.

Automated discovery reduces configuration drift and ensures the workflow always operates against the current infrastructure state.

Each discovered instance is evaluated before being added to the execution plan.

```
((.venv) )
user@hompson-Desktop MINGW64 /c:/TechPathway/Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

=====
EC2 Infrastructure Standardization Automation
=====

Configuration Loaded Successfully

AWS Region      : us-east-1
AWS Profile     : default

Application     : inventory-platform
Environment     : production

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances  : 15

Dry Run        : True
Canary Instance : inventory-api-05
Batch Size     : 3
Rollback Enabled : True

Configuration validation successful.

Connecting securely to AWS...
2026-07-30 20:05:02,301 | INFO | Successfully authenticated with AWS.
AWS connection successful.
AWS Account ID      : 202937691499
EC2 Region          : us-east-1

Discovering EC2 instances using configured tags...
2026-07-30 20:05:03,149 | INFO | Discovered 15 matching EC2 instances.

Discovery Summary
-----
Expected Instances : 15
Discovered Instances : 15
Discovery Status   : PASS
-----

Name                Role    Type    State    Availability Zone
-----
inventory-api-01     api     t3.micro running us-east-1a
inventory-api-02     api     t3.micro running us-east-1b
inventory-api-03     api     t3.micro running us-east-1a
inventory-api-04     api     t3.micro running us-east-1b
inventory-api-05     api     t3.micro running us-east-1a
inventory-cache-01    cache   t3.micro running us-east-1a
inventory-cache-02    cache   t3.micro running us-east-1b
inventory-report-01   reporting t3.micro running us-east-1a
inventory-report-02   reporting t3.micro running us-east-1b
```

Figure 5. Dynamic discovery of the target EC2 fleet before execution.

Safety Validation

Before modifying any instance, I perform several validation checks to confirm that each resource is eligible for change.

These checks include verifying the current instance type, power state, AWS API accessibility, and other operational prerequisites required for a successful resize.

The objective is simple: fail before making changes whenever the environment does not meet the expected conditions.

```
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
$ python resize_instances.py

Source Instance Type : t3.micro
Target Instance Type : t3.small
Expected Instances   : 15

Dry Run             : True
Canary Instance     : inventory-api-05
Batch Size          : 3
Rollback Enabled    : True

Configuration validation successful.

Connecting securely to AWS...
2026-07-30 16:42:03,989 | INFO | Successfully authenticated with AWS.
AWS connection successful.
AWS Account ID      : 2025
EC2 Region          : us-east-1

Discovering EC2 instances using configured tags...

Discovery Summary
-----
Expected Instances   : 15
Discovered Instances : 0
Discovery Status     : FAILED
-----

Fleet Validation
-----
Validation Status    : FAILED
-----

The automation cannot continue:
- Expected 15 instances, but discovered 0.
- Configured canary instance 'inventory-api-05' was not discovered.

No EC2 changes were performed.
-----
((.venv) )
user@Thompson-Desktop MINGW64 /c:/TechPathway Projects/python-lab/ec2-infrastructure-standardization/automation
```

Figure 6. Safety validation confirms that the environment is ready before infrastructure modifications begin.

Canary Deployment Strategy

Updating an entire fleet simultaneously may reduce execution time, but it also significantly increases operational risk.

Instead, I implemented a canary deployment strategy that upgrades a single EC2 instance first.

Only after the canary instance completes successfully does the automation continue with the remaining fleet.

This approach limits the blast radius of unexpected failures and provides confidence before broader deployment.

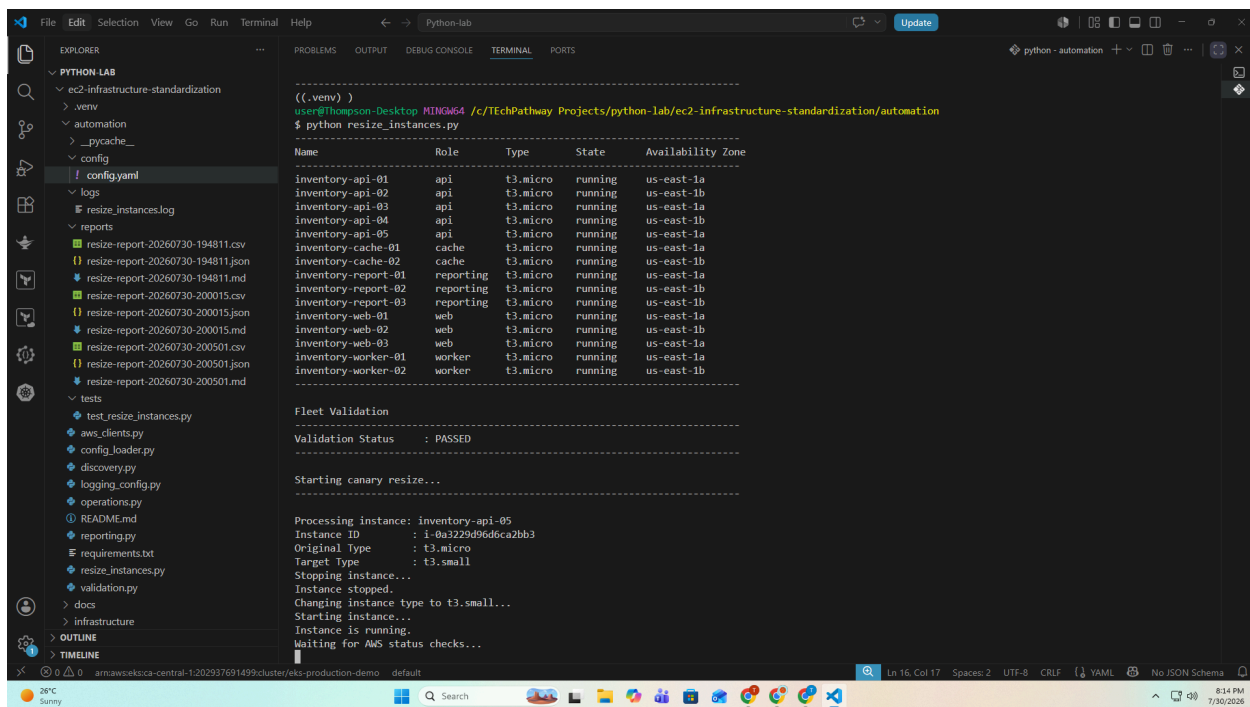


Figure 7. Canary execution validating the workflow on a single EC2 instance before fleet-wide deployment.

Controlled Batch Execution

Following successful canary validation, the remaining instances are processed in controlled batches rather than simultaneously.

Batch execution provides several operational advantages:

- Reduced blast radius
- Easier troubleshooting
- Lower API throttling risk
- Simpler rollback planning
- Improved execution visibility

Although sequential batches increase total execution time, I intentionally accepted this trade-off to improve operational safety.

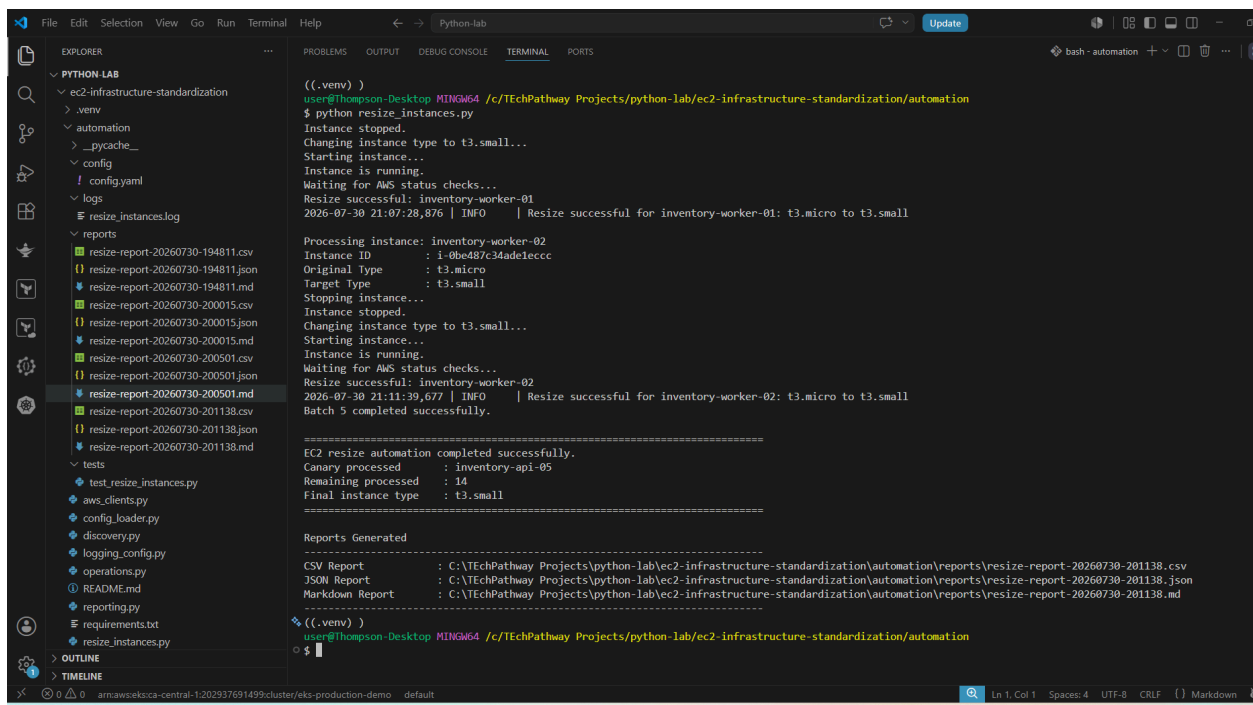


Figure 8. Successful batch execution across the remaining EC2 instances.

Reporting and Audit Trail

Every execution generates structured reports containing validation results, execution summaries, timestamps, and infrastructure changes.

Producing detailed execution reports improves troubleshooting while providing an audit trail for future operational reviews.

Rather than relying solely on console output, the automation records execution artifacts that can be reviewed after completion.

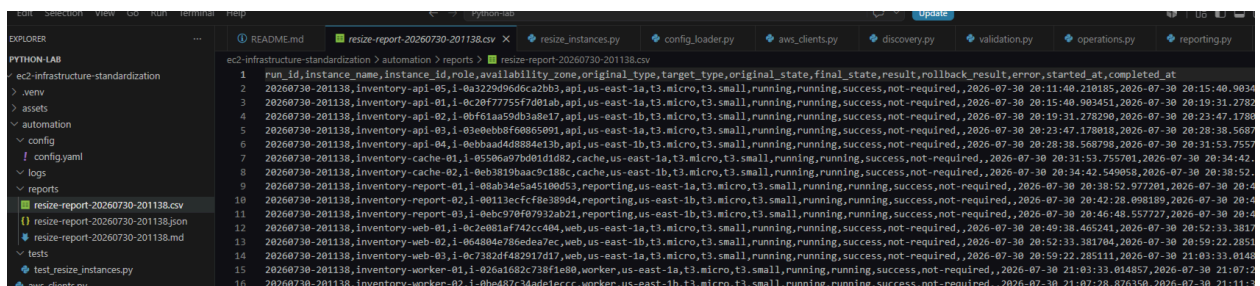
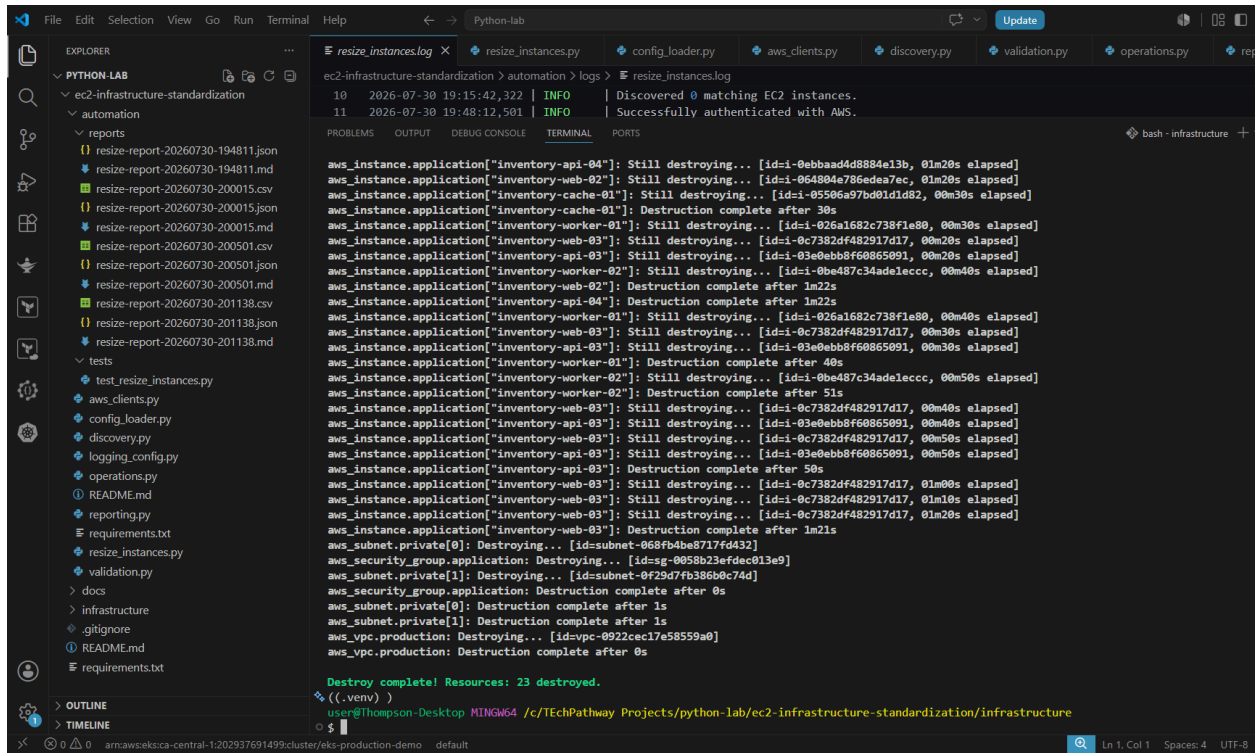


Figure 9. Generated execution reports providing traceability and operational visibility.

Infrastructure Cleanup

To avoid unnecessary AWS costs and maintain a clean development environment, the infrastructure can be removed entirely using Terraform after testing has been completed.

Keeping provisioning and cleanup fully automated ensures the environment can be recreated consistently whenever additional testing is required.



```
ec2-infrastructure-standardization > automation > logs > F resize_instances.log
10 2026-07-30 19:15:42,322 | INFO | Discovered 0 matching EC2 instances.
11 2026-07-30 19:48:12,501 | INFO | Successfully authenticated with AWS.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - infrastructure +

aws_instance.application["inventory-api-04"]: Still destroying... [id=i-0ebbbaad4d8884a13b, 01m20s elapsed]
aws_instance.application["inventory-web-02"]: Still destroying... [id=i-064804e786ede7ec, 01m20s elapsed]
aws_instance.application["inventory-cache-01"]: Still destroying... [id=i-05506a97bd01d1d82, 00m30s elapsed]
aws_instance.application["inventory-worker-01"]: Destruction complete after 30s
aws_instance.application["inventory-worker-01"]: Still destroying... [id=i-026a1682c738f1e80, 00m30s elapsed]
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 00m20s elapsed]
aws_instance.application["inventory-api-03"]: Still destroying... [id=i-03e0ebb8f60865091, 00m20s elapsed]
aws_instance.application["inventory-worker-02"]: Still destroying... [id=i-0be487c34adeleccc, 00m40s elapsed]
aws_instance.application["inventory-web-02"]: Destruction complete after 1m22s
aws_instance.application["inventory-api-04"]: Destruction complete after 1m22s
aws_instance.application["inventory-worker-01"]: Still destroying... [id=i-026a1682c738f1e80, 00m40s elapsed]
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 00m30s elapsed]
aws_instance.application["inventory-api-03"]: Still destroying... [id=i-03e0ebb8f60865091, 00m30s elapsed]
aws_instance.application["inventory-worker-01"]: Destruction complete after 40s
aws_instance.application["inventory-worker-02"]: Still destroying... [id=i-0be487c34adeleccc, 00m50s elapsed]
aws_instance.application["inventory-worker-02"]: Destruction complete after 51s
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 00m40s elapsed]
aws_instance.application["inventory-api-03"]: Still destroying... [id=i-03e0ebb8f60865091, 00m40s elapsed]
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 00m50s elapsed]
aws_instance.application["inventory-api-03"]: Still destroying... [id=i-03e0ebb8f60865091, 00m50s elapsed]
aws_instance.application["inventory-api-03"]: Destruction complete after 50s
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 01m00s elapsed]
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 01m10s elapsed]
aws_instance.application["inventory-web-03"]: Still destroying... [id=i-0c7382d4f482917d17, 01m20s elapsed]
aws_instance.application["inventory-web-03"]: Destruction complete after 1m21s
aws_subnet.private[0]: Destroying... [id=subnet-068fb4be8717fd432]
aws_security_group.application: Destroying... [id=sg-0058b23efdec013e9]
aws_subnet.private[1]: Destroying... [id=subnet-0f29d7fb386b0c74d]
aws_security_group.application: Destruction complete after 0s
aws_subnet.private[0]: Destruction complete after 1s
aws_subnet.private[1]: Destruction complete after 1s
aws_vpc.production: Destroying... [id=vpc-0922cec1e58559a0]
aws_vpc.production: Destruction complete after 0s

Destroy complete! Resources: 23 destroyed.
❖ (..venv)
user@Thompson-Desktop MINGW64 /c:/TechPathway/Projects/python-lab/ec2-infrastructure-standardization/infrastructure
```

Figure 10. Infrastructure cleanup using Terraform after successful validation and testing.